

অপারেটিং সিস্টেমের ভূমিকা

(Introduction to Operating Systems)

প্রফেসর চেস্টার রেবেরো

(Prof. Chester Rebeiro)

কম্পিউটার বিজ্ঞান ও প্রকৌশল বিভাগ

(Department of Computer Science and Engineering),

ইন্ডিয়ান ইনস্টিটিউট অফ টেকনোলজি, মাদ্রাজ

(Indian Institute of Technology, Madras)

সপ্তাহ – 06

বক্তৃতা – 29

সিমাফোরস

(Semaphores)

হ্যালো এই ভিডিওটিতে আমরা সিমাফোরস নামে পরিচিত আরেকটি synchronization primitive দেখতে পাব।
আমরা প্রেরণাদায়ক উদাহরণ দিয়ে শুরু করব এবং তারপর semaphores অ্যাপ্লিকেশন প্রদর্শন করব।

(স্লাইড সময় : 00:35 দেখুন)

সুতরাং আসুন শুরু করা যাক Producer-Consumer Problem হিসাবে পরিচিত একটি জনপ্রিয় উদাহরণ দিয়ে।
সুতরাং এটি Bounded Buffer Problem হিসাবেও পরিচিত। মূলত, এখানে দুটি প্রসেস আছে; একটি প্রক্রিয়া producer হিসাবে পরিচিত এবং অন্যটি consumer হিসাবে পরিচিত হয়। এখন producer এবং consumer একটি সীমিত buffer share করে। এটি একটি স্বাভাবিক buffer এবং এটির size N অর্থাৎ N data element এটিতে সংরক্ষণ করা যেতে পারে। সুতরাং এই বিশেষ ক্ষেত্রে উদাহরণস্বরূপ, 6 টি element বাফার এর মধ্যে সংরক্ষণ করা যাবে। এখন producer data উৎপাদন করে সুতরাং উদাহরণস্বরূপ, এটি একটি data acquisition module হতে পারে যেমন তাপমাত্রা, চাপ ইত্যাদি হিসাবে তথ্য সংগ্রহ করে, তাই এই data buffer এর মধ্যে ঢুকিয়ে দেওয়া হয়। এখন অন্যদিকে, consumer buffer থেকে data বের করে নেয় এবং তারপর data গুলি প্রক্রিয়া করে। সুতরাং উদাহরণস্বরূপ, এই consumer প্রক্রিয়া সম্ভবত producer data এর উপর কিছু বিশ্লেষণ

গণনা করতে পারে। তাই, সবকিছুই বেশ ভালভাবে কাজ করবে, যা producer data উৎপন্ন করে, buffer এ রাখে এবং অন্যদিকে consumer buffer থেকে data ব্যবহার করতে শুরু করে। উদাহরণস্বরূপ data দিয়ে কিছু গণনা করা হয়। উদাহরণস্বরূপ, যখন producer এর তুলনায় consumer খুব ধীর হবে তখন সমস্যা হবে। এই ক্ষেত্রে producer consumer এর তুলনায় দ্রুততর হারে প্রচুর পরিমাণে data produce করবে এবং সেইজন্য খুব দ্রুত buffer পূর্ণ হবে। তাই, producer পরবর্তীতে কি করবে? আরেকটি সমস্যা হতে পারে যেখানে producer এর তুলনায় consumer খুবই দ্রুত। এবং সেইজন্য, এটি খুব দ্রুত buffer এর সমস্ত ডেটা consume করতে পারে, এবং buffer খালি হবে। সুতরাং, consumer এর পরবর্তী কি করা উচিত? সুতরাং producer এবং consumer মধ্যে একটি সুসংহত (synchronization) প্রক্রিয়া প্রয়োজন। মূলত, যখন buffer খালি হয় তখন producer এর দ্বারা buffer data পূরণ না হওয়া পর্যন্ত consumer কে অপেক্ষা করতে হবে। একইভাবে, যখন buffer পূর্ণ হয় তখন producer কে অপেক্ষা করতে হবে যতক্ষণ না consumer buffer থেকে data বের করে নেয়।

(স্লাইড সময় : 03:25 দেখুন)

সুতরাং এটি সাধারণ producer and consumer code, এবং আমরা Mutexes ব্যবহার করে producer-consumer সমস্যা সমাধান করার চেষ্টা করব। মূলত, 3টি mutex ব্যবহার করা হয়; empty, full, এবং একটি Mutex যাকে mutex বলা হয়। সুতরাং, producer code মূলত একটি item উৎপাদন করে, item buffer এর মধ্যে যোগ হয় এবং count বৃদ্ধি করে, count++। Consumer code item টি সরিয়ে ফেলবে, একটি count হ্রাস করবে এবং তারপর item টি consume করবে। এখন অস্থির পরিস্থিতির যন্ত্র নেওয়ার জন্য, যখন buffer পূর্ণ বা buffer খালি থাকে তখন আমরা এই mutex ব্যবহার করছি; empty এবং full। মূলত item insert করার আগে producer যদি N এর সমান গণনা করে তবে তাতে চেক করা যাচ্ছে buffer পূর্ণ কিনা। এবং যদি buffer পূর্ণ হয়, তাহলে এই বিশিষ্ট mutex এ empty তে sleep করে। এবং অন্যদিকে, consumer যদি count N-1 এর সমান হয় তবে এর মানে হল যে এটি একটি পূর্ণ buffer থেকে একটি item সরানো হয়েছে। যদি তাই হয় তাহলে এটি empty কে wake up করবে। তাই এই wake up producer কে তার sleep থেকে জাগিয়ে দেবে এবং তারপর producer item বাফারে insert এবং count বৃদ্ধি করতে পারে। অন্য দিকে, যদি consumer জানতে পারে যে buffer খালি আছে, এটি count 0 এর সমান, তাহলে এই mutex full এ sleep করবে। সুতরাং, এটি producer প্রযোজক থেকে একটি wake up পাওয়া পর্যন্ত এই mutex block হবে। মূলত, যদি producer খুঁজে পায় যে যখন item insert হয় এবং count বৃদ্ধি করে যে buffer এর মধ্যে ঠিক 1 টি আইটেম উপস্থিত রয়েছে তখন সে consumer কে একটি wake up signal পাঠাবে, তাই wake up সম্পূর্ণ। এবং তাই, এই wakeup full consumer কে আনলক করে এবং এটি ready queue এর মধ্যে ফিরে করা হবে এবং এটি consumer কে চালানোর অনুমতি দেবে, এবং ওই item টি remove এবং তারপর আইটেমটি consume হবে। তাই item

remove এর পর তার সংখ্যা 0 তে চলে যায়। এখন এই empty এবং full mutex ছাড়াও তৃতীয় mutex রয়েছে যা ব্যবহার করা হয়। সুতরাং, এই mutex মূলত buffer access protection বা synchronize করার জন্য ব্যবহৃত হয়। সুতরাং, একটি item insert করার আগে এবং count গণনা বৃদ্ধি করার পর, producer এর mutex কে লক করা প্রয়োজন এবং আনলকিং হয় item push এর পরে এবং count বৃদ্ধি করা হয়। অন্য দিকে, buffer টি item remove এর জন্য access করার আগে এবং count হ্রাস করা হয়। মূলত আপনি লক্ষ্য করেন যে count এবং buffer দুটি প্রক্রিয়ায় producer এবং consumer এ share করা হয়। এবং তাই, mutex এই buffer এবং count মান access synchronize করতে সাহায্য করবে, তাই এই সমাধান 3 টি mutex এর সঙ্গেও ভালো কাজ করবে বলে মনে হয়। আমরা দেখব যে কোনও নির্দিষ্ট শর্তের অধীনে producer এবং consumer infinitely কোন progress ছাড়াই block করবে। তাই এই শর্তটি যে এই বিশেষ লাইনের উপর ভিত্তি করে, count 0 হলে গণনা যদি প্রকৃতপক্ষে দুটি step গঠিত হয় যা atomic নয়। প্রথম step হলো মেমরিতে সংরক্ষিত count এর মান প্রসেসরের একটি রেজিস্টারে লোড হবে, এবং দ্বিতীয় step হল যখন count 0 কিনা তা চেক করা হয়। সুতরাং আসুন আমরা এই সমস্যাটি দেখি যে এই বিশেষ statement non-atomic।

(স্লাইড সময় : 08:12 দেখুন)

তাই আসুন আমরা বলি, consumer ভোক্তা প্রথমে নির্বাহ শুরু করে এবং একটি empty buffer এর মাধ্যমে এটি চালানো শুরু করে। সুতরাং এটি একটি মেমরি অবস্থান থেকে count এর মান একটি register এ পড়ে, কারণ আমরা মনে করি যে এই প্রাথমিক অবস্থা তাই হিসাবের মান যা রেজিস্টারে রাখা হবে তা 0 হতে হবে। তাহলে আমরা বলব যে একটি context switch ঘটেছে এবং producer চালানো হয়েছে। এখন producer একটি item উৎপাদন করে, count 1 বৃদ্ধি করে, এবং তারপর item টি buffer এর মধ্যে insert করে। এখন এটি চালানোর পরে, আমাদের একটি context switch আবার আছে বলে, এটি consumer যে বিন্দুতে stop ছিল সেখান থেকে চালানো হয়। এখন আমরা জানি যে এটি ইতিমধ্যে 0 এর মানের সঙ্গে পূর্বে register এ লোড করেছে। এখন এটি পরীক্ষা করবে যে count 0 কিনা, এই ক্ষেত্রে আটা সত্য এবং সেইজন্য consumer অপেক্ষা করে। মূলত, এটি producer থেকে একটি সংকেত পাওয়া পর্যন্ত অপেক্ষা করে। যাইহোক, count এর প্রকৃত মান 1, কারণ producer একটি item বাফারের মধ্যে insert করেছে। এবং এইভাবে আমরা দেখতে পাওয়া যায় যে হারিয়ে যাওয়া wakeup ও হয়। Consumer একটি wake up সংকেত হারিয়েছে যা producer প্রয়োজক পাঠিয়েছে। এখন buffer মধ্যে আরও item গুলি insert করতে producer কে থামানো যাবে না। সুতরাং, অবশেষে পুরো বাফারটি full এবং producer তারপর consumer দ্বারা কিছু item remove জন্য অপেক্ষা করবে। তবে, এটা ঘটবে না কারণ consumer নিজে অপেক্ষা করছে। এইভাবে, আমরা একটি producer একটি item insert করার জন্য consumer এর জন্য অপেক্ষারত আছে, যখন consumer ও অপেক্ষা করে আছে কারণ এটি wakeup মিস

করেছে। সুতরাং, এটি একটি নির্দিষ্ট state যেখানে producer এবং consumer উভয় অসীমভাবে অপেক্ষা করবে। তাই আমরা দেখতে পাই যে তিনটি mutexes ব্যবহার করে producer-consumer সমস্যা সমাধান করা যাবে না।

(স্লাইড সময় 10:39 দেখুন)

আসুন দেখি আরেকটি primitive যেটি semaphore নামে পরিচিত। তাই এই semaphore আরেকটি synchronization primitive এটি Dijkstra দ্বারা প্রস্তাবিত হয় 1965 তে। এবং semaphores এ আছে দুইটি function down এবং up যা আমরা অনুমান করি atomic বলে। সুতরাং এই দুটি function যা atomic করা প্রয়োজন। সুতরাং একটি shared memory location যা S হিসাবে বলা হয় এবং যেটা আছে down ফাংশানে, while লুপ পরীক্ষা করবে যে S 0 এর সমান বা কম হয় কিনা। সুতরাং, যতক্ষণ S এর মান 0 এর সমান বা কম হয় এই লুপটি চালাতে হবে। যখন S 0 এর চেয়ে বড় মান গ্রহণ করে, তখন লুপ ভেঙে যাবে এবং সেই memory location S এর মানটি 1 দ্বারা হ্রাস হবে। up ফাংশানে যা atomic memory location S এর মান 1 বাড়ানো হয়। সুতরাং, down এবং up function গুলিকে মাঝে মাঝে P এবং V ফাংশন হিসাবে ডাকা হয়, যেগুলি Dutch নাম। এবং আমাদের semaphore এর দুটি ভিন্ন রূপ হতে পারে; আমাদের একটি blocking semaphore এবং একটি non-blocking থাকতে পারে। সুতরাং, এখানে একটি non-blocking semaphore দেখানো হয়েছে, মূলত এটি একটি while loop যা একটি busy waiting spinlock এর মত। অন্যদিকে আমরা একটি ছোট সংশোধন করতে পারি এবং একটি blocking semaphore তৈরি করতে পারি, যেখানে এই নির্দিষ্ট statement টি block অবস্থায় প্রক্রিয়ায় রূপ নেয়, যখন up সংকেতটি প্রক্রিয়াটি জাগবে। সুতরাং যদি S এর মান শুরুতে সেট করা হয় 1 তারপর একটি blocking semaphore একটি mutex এর অনুরূপ, একটি non-unblocking semaphore একটি spinlock এর অনুরূপ। তাই এখন, দেখি কিভাবে আমরা producer-consumer সমস্যা সমাধানের জন্য semaphore ব্যবহার করতে পারি।

(স্লাইড সময় 12:55 দেখুন)

তাই সমস্যা সমাধানের জন্য আমাদের দুই semaphores প্রয়োজন; একটি full হিসাবে পরিচিত হয়। সুতরাং যখন আমি দুই semaphore এর কথা বলছি এটি দুটি memory location মানে আমাদের এই S দ্বারা নির্দিষ্ট এখানে। সুতরাং আমাদের দুটি memory location বা দুটি semaphores আছে, full এবং empty। সুতরাং, full এর প্রাথমিক সংখ্যা 0 দেওয়া হয়, যখন empty এর প্রাথমিক মান N দেওয়া হয়, যেখানে N এখানে buffer size। Semaphore full buffer এর ভরা ব্লকের সংখ্যা নির্দেশ করে, যখন semaphore empty খালি buffer এর খালি ব্লকের সংখ্যা নির্দেশ করে। এই বিশেষ ক্ষেত্রে, যেখানে N 6 এর সমান, full 4 এর মান থাকবে কারণ 4 টি full ব্লক এবং empty 2 এর মান থাকবে কারণ 2 খালি ব্লক রয়েছে। সুতরাং, producer এবং consumer execute

কার্যকর হওয়ার ঠিক আগে প্রাথমিক অবস্থাগুলি full 0 এর সমান এবং empty N এর সমান হবে, কারণ কোনও buffer এ data নেই; মূলত, কারণ buffer খালি আছে।

(স্লাইড সময় 14:13 দেখুন)

সুতরাং আসুন দেখি কিভাবে semaphores ব্যবহার করা হয়। তাই আসুন producer টি দেখি, তাই producer একটি item তৈরি করে এবং আমরা এই বিশেষ উদাহরণটি নিতে পারি যেখানে একটি full 4 এবং empty 2, এবং তারপর যখন item টি তৈরি করা হয় তখন সেটি semaphore down কে invoke করে। তাই আমরা down semaphore empty semaphore কে নীচে নামাবে, empty 2 থেকে 1 এ যেতে হবে। সুতরাং, এটি একটি atomic অপারেশন।

(স্লাইড সময় 14:52 দেখুন)

তারপর producer এর পরবর্তী পদক্ষেপ একটি item insert করা হয়, তাই একটি নতুন item buffer এর মধ্যে ঢোকানো হয়। তারপর semaphore full পূর্ণ 5 এর মান পাবে।

(স্লাইড সময় 15:08 দেখুন)

একইভাবে, consumer এর অংশ নিম্নরূপ:। প্রথমে down full হবে, তাই full এর মান 5 থেকে 4 যেতে হবে।

(স্লাইড সময় 15:25 দেখুন)

তারপর একটি item buffer থেকে মুছে ফেলা হয় এবং empty মান up হিসাবে সেট হয়। সুতরাং, empty এর মান 2 হতে হবে, এবং তারপর consumer নির্দিষ্ট item consume করবে।

(স্লাইড সময় 15:41 দেখুন)

এখন আসুন আমরা producer এবং consumer এবং একটি full buffer ক্ষেত্রে দেখতে পাই। সুতরাং আমরা মনে করি যে buffer full হয়, তাই এই ক্ষেত্রে একটি full মান আছে 6 যা N এর সমান, যখন empty তে 0 এর মান আছে ইঙ্গিত করে যে কোনও empty ব্লক buffer এ নেই, এবং full 6 এর ইঙ্গিত দেয় যে buffer এ 6 টি full ব্লক রয়েছে। তাই স্বাভাবিকভাবে producer একটি item উৎপাদন করা হবে এবং তারপর এটি empty হ্রাস করবে। এখন আপনি দেখতে পাবেন যে empty তে 0 এর মান আছে। সুতরাং যদি আপনি semaphore down ফাংশনে ফিরে যান, তবে এটি while statement এর মধ্যে ক্রমাগত চলতে থাকবে। তাই, producer এই down semaphore এ blocked বা waiting হবে। এখন, পরে যখন consumer চালানো শুরু হবে তখন এটি down

full করবে; তাই ফলাফল হিসাবে, এটি একটি বিশেষ item consume করবে তাই এটি full কে down করবে তাই full 5 হবে। এবং তারপর এটি একটি item remove করবে এবং empty up হবে।

(স্লাইড সময় 17:00 দেখুন)

তাই empty এখন 1 এ সেট করা হয়, এবং অবশ্যই item টি consume করতে যাচ্ছে। এখন empty 1 এ সেট করলে semaphore down wakeup হবে বা semaphore লুপ ভেঙ্গে যাবে এবং তারপর producer empty কে 0 তে সেট করবে এবং item buffer এর মধ্যে insert করবে এবং তারপর full মান আবার 6 এ সেট করা হয়।

(স্লাইড সময় 17:31 দেখুন)

সুতরাং এই ভাবে, semaphores full এবং empty producer-consumer সমস্যা সমাধান করতে ব্যবহৃত হয়।

(স্লাইড সময় 17:49 দেখুন)

একই রকম বিশ্লেষণ তৈরি করা যেতে পারে যখন buffer খালি থাকে। এই ক্ষেত্রে, full এবং empty মান যথাক্রমে 0 এবং 6।

(স্লাইড সময় 18:03 দেখুন)

সুতরাং, producer and consumer কোডের মধ্যে আমরা যা করিনি, আমরা এই বিশেষ buffer access synchronization করিনি। ফলস্বরূপ এটি সম্ভব হতে পারে যে উৎপাদক buffer item ঢোকাতে পারে এবং একই সময়ে গ্রাহক buffer থেকে একটি item সরিয়ে ফেলতে পারে। তাই এমন জিনিস প্রতিরোধ করার জন্য আমরা buffer access synchronize করার জন্য একটি mutex ব্যবহার করি। সুতরাং এই নির্দিষ্ট buffer access করার আগে, একটি producer এবং পাশাপাশি consumer mutex লক করতে হবে, এবং buffer access এর পরে mutex আনলক করা প্রয়োজন। এই mutex এর ফলে আমরা নিশ্চিত হতে পারি যে এই দুটি প্রসেসের মধ্যে একটি মাত্র এই বিশেষ critical section এ execute করছে, যেকোনো মুহূর্তে buffer access করছে। সুতরাং, এখন producer-consumer সমস্যাগুলির অন্য কিছু রূপ আছে এবং বিভিন্ন ধরনের scheme আছে যার মধ্যে semaphore এর এই বিভিন্ন সমস্যার সমাধান করতে ব্যবহার করা যেতে পারে। কিন্তু, এই বিশেষ কোর্সের জন্য আমাদের এই বিশেষ উদাহরনে থামাতে হবে। ধন্যবাদ।